

List of the key Concepts, Principles, and Components of OPP.

The Four Pillars of OOP

1. Encapsulation:

- **Definition:** Bundling data and methods that operate on that data into a single unit (Class).
- **Key Feature:** Data Hiding. Restricting direct access to some of an object's components.
- **Benefit:** Protects internal state from unintended interference and misuse.

The Four Pillars of OOP

1. Encapsulation

2. Abstraction:

- **Definition:** Hiding complex implementation details and showing only the necessary features of an object.
- **Key Feature:** Focusing on what an object does rather than how it does it.
- **Benefit:** Reduces programming complexity and effort.

The Four Pillars of OOP

1. Encapsulation

2. Abstraction

3. Inheritance:

- **Definition:** A mechanism where a new class (Child/Subclass) derives properties and behaviors from an existing class (Parent/Superclass).
- **Key Feature:** Promotes code reusability.
- **Types:** Single, Multiple, Multilevel, Hierarchical, Hybrid.

The Four Pillars of OOP

1. Encapsulation

2. Abstraction

3. Inheritance

4. Polymorphism:

- **Definition:** The ability of an object to take on many forms.
- **Key Feature:** Allows methods to do different things based on the object acting on them.
- Types:
 - Compile-time (Overloading): Same method name, different parameters.
 - Runtime (Overriding): Same method name and parameters in parent and child class, but different implementation.

Basic Building Blocks

- **Class:** A blueprint or template from which objects are created. It defines the structure (attributes) and behavior (methods).
- **Object:** An instance of a class. It is the actual entity in memory that holds data.
- **Attributes (Fields/Properties):** Variables defined within a class that hold the state of the object.
- **Methods:** Functions defined within a class that define the behavior of the object.
- **Constructor:** A special method automatically called when an object is created. Used to initialize the object's state.
- **Destructor:** A special method called when an object is destroyed (used for cleanup, mainly in languages like C++).

Access Modifiers (Visibility)

Controls the visibility of classes, methods, and variables.

- **Public:** Accessible from anywhere.
- **Private:** Accessible only within the same class.
- **Protected:** Accessible within the same class and its subclasses (children).
- **Internal/Package-Private:** Accessible only within the same package or assembly (language-dependent).

Class Relationships

How objects interact with one another.

- **Association:** A relationship where two objects are related (e.g., Teacher and Student).
- **Aggregation ("Has-A"):** A weak relationship where one object contains another, but the contained object can exist independently (e.g., Department has Professors; if Department closes, Professors still exist).
- **Composition ("Part-Of"):** A strong relationship where one object owns another, and the contained object cannot exist without the owner (e.g., House has Rooms; if House is destroyed, Rooms are destroyed).
- **Dependency:** A relationship where a change in one class affects the other (usually via method parameters).

Advanced OOP Concepts

- **Abstract Class:** A class that cannot be instantiated on its own. It may contain abstract methods (no body) that must be implemented by subclasses. Used for partial abstraction.
- **Interface:** A contract that defines a set of methods a class *must* implement. It contains no implementation logic (traditionally). Used for full abstraction.
- **Static Members:** Belong to the class itself rather than any specific object instance. Shared across all objects.
- **Coupling:** The degree of interdependence between software modules. **Low coupling** is desired in OOP.
- **Cohesion:** The degree to which the elements inside a module belong together. **High cohesion** is desired in OOP.

SOLID Principles

Five design principles intended to make software designs more understandable, flexible, and maintainable.

- **S - Single Responsibility Principle:** A class should have one, and only one, reason to change.
- **O - Open/Closed Principle:** Entities should be open for extension but closed for modification.
- **L - Liskov Substitution Principle:** Subtypes must be substitutable for their base types without breaking the application.
- **I - Interface Segregation Principle:** Clients should not be forced to depend on interfaces they do not use.
- **D - Dependency Inversion Principle:** Depend on abstractions, not on concretions.

Benefits of OOP

- **Modularity:** Code is organized into distinct objects.
- **Reusability:** Inheritance allows code to be reused.
- **Maintainability:** Easier to update and debug specific parts without affecting the whole.
- **Scalability:** Easier to manage large codebases.
- **Security:** Data hiding protects sensitive information.

Drawbacks of OOP

- **Verbosity:** Requires more lines of code compared to procedural programming.
- **Performance:** Can be slightly slower due to abstraction layers and dynamic dispatch (though negligible on modern hardware).
- **Complexity:** Steep learning curve; bad design can lead to overly complex class hierarchies.
- **Not Universal:** Not suitable for all problems (e.g., simple scripts or high-performance computing tasks might be better suited for procedural or functional programming).

Popular OOP Languages

- **Java:** Purely object-oriented (everything must be in a class).
- **C++:** Multi-paradigm (supports OOP, procedural, generic).
- **Python:** Multi-paradigm (supports OOP, functional, procedural).
- **C#:** Designed for the .NET framework, heavily OOP.
- **Ruby:** Purely object-oriented.
- **PHP:** Widely used for web, supports OOP.

BAD CODE Examples for SOLID Refactoring

- *Now the fun part 🤖*
- *These are intentionally bad designs for students to fix.*

Single Responsibility Principle (SRP) Violation

```
class StudentService
{
    public void SaveStudent(Student student)
    {
        // Save to database
    }

    public void SendEmail(Student student)
    {
        // Send welcome email
    }

    public void GenerateReport(Student student)
    {
        // Generate PDF report
    }
}
```

Single Responsibility Principle (SRP) Violation

```
class StudentService
{
    public void SaveStudent(Student student)
    {
        // Save to database
    }

    public void SendEmail(Student student)
    {
        // Send welcome email
    }

    public void GenerateReport(Student student)
    {
        // Generate PDF report
    }
}
```

Problems

- Database logic
- Email logic
- Reporting logic

ALL inside one class.

Single Responsibility Principle (SRP) Violation

```
class StudentService
{
    public void SaveStudent(Student student)
    {
        // Save to database
    }

    public void SendEmail(Student student)
    {
        // Send welcome email
    }

    public void GenerateReport(Student student)
    {
        // Generate PDF report
    }
}
```

Problems

- Database logic
- Email logic
- Reporting logic

ALL inside one class.

Refactoring Goal

StudentService
must split into:

- StudentRepository
- EmailService
- ReportService
- StudentService

Open/Closed Principle (OCP) Violation

```
class PaymentProcessor
{
    public void Process(string type)
    {
        if (type == "Cash")
        {
            Console.WriteLine("Processing cash");
        }
        else if (type == "Card")
        {
            Console.WriteLine("Processing card");
        }
        else if (type == "Bank")
        {
            Console.WriteLine("Processing bank
transfer");
        }
    }
}
```

Open/Closed Principle (OCP) Violation

```
class PaymentProcessor
{
    public void Process(string type)
    {
        if (type == "Cash")
        {
            Console.WriteLine("Processing cash");
        }
        else if (type == "Card")
        {
            Console.WriteLine("Processing card");
        }
        else if (type == "Bank")
        {
            Console.WriteLine("Processing bank transfer");
        }
    }
}
```

Problem

Every new payment type requires modifying the class.

Open/Closed Principle (OCP) Violation

```
class PaymentProcessor
{
    public void Process(string type)
    {
        if (type == "Cash")
        {
            Console.WriteLine("Processing cash");
        }
        else if (type == "Card")
        {
            Console.WriteLine("Processing card");
        }
        else if (type == "Bank")
        {
            Console.WriteLine("Processing bank transfer");
        }
    }
}
```

Problem

Every new payment type requires modifying the class.

Refactoring Goal

Use polymorphism:

```
interface IPayment
{
    void Process();
}
```

Then:

- CashPayment
- CardPayment
- BankPayment

No if/else allowed.

Liskov Substitution Principle (LSP) Violation

```
class Bird
{
    public virtual void Fly()
    {
        Console.WriteLine("Flying");
    }
}

class Penguin : Bird
{
    public override void Fly()
    {
        throw new NotImplementedException();
    }
}
```

Liskov Substitution Principle (LSP) Violation

```
class Bird
{
    public virtual void Fly()
    {
        Console.WriteLine("Flying");
    }
}

class Penguin : Bird
{
    public override void Fly()
    {
        throw new NotImplementedException();
    }
}
```

Problem

Penguin cannot fly → breaks substitution.

Liskov Substitution Principle (LSP) Violation

```
class Bird
{
    public virtual void Fly()
    {
        Console.WriteLine("Flying");
    }
}

class Penguin : Bird
{
    public override void Fly()
    {
        throw new NotImplementedException();
    }
}
```

Problem

Penguin cannot fly → breaks substitution.

Refactoring Goal

Split into:

- Bird
- FlyingBird
- Penguin
- Eagle

Or use interfaces:

- IFlyable

Interface Segregation Principle (ISP) Violation

```
interface IWorker
{
    void Work();
    void Eat();
}
```


Interface Segregation Principle (ISP) Violation

```
interface IWorker
{
    void Work();
    void Eat();
}
```

Problem:

Robot must implement Eat() even if it doesn't eat.

Interface Segregation Principle (ISP) Violation

```
interface IWorker
{
    void Work();
    void Eat();
}
```

Problem:

Robot must implement Eat()
even if it doesn't eat.

Refactor Into:

- Iworkable
- IEatable

Dependency Inversion Principle (DIP) Violation

```
class StudentService
{
    private MySQLDatabase database = new
    MySQLDatabase();

    public void Save(Student student)
    {
        database.Save(student);
    }
}
```

Dependency Inversion Principle (DIP) Violation

```
class StudentService
{
    private MySQLDatabase database = new
    MySQLDatabase();

    public void Save(Student student)
    {
        database.Save(student);
    }
}
```

Problem

High-level class depends on concrete class.

Dependency Inversion Principle (DIP) Violation

```
class StudentService
{
    private MySQLDatabase database = new
        MySQLDatabase();

    public void Save(Student student)
    {
        database.Save(student);
    }
}
```

Problem

High-level class depends on
concrete class.

Refactor Goal

```
interface IDatabase
{
    void Save(Student student);
}
```

Inject via constructor.